Validation Notes on the Problem Formulation

Contents

1 Internal-consistency review **2**
- 1.1 [G14] Congestion sensitivity in routing was decentralised-only, which breaks the cross-architecture comparison . . . 5
- 1.2 [G15] Computational cost was promised in the new Introduction but never formalised 5
- 1.3 [G19] Three local radii (d_{obs}, r_{cng}, r_{com}) and `[G14]`'s general $\delta(S, t; \text{excl})$ collapsed into one field of view . . . 7
- 1.4 [G20] Decentralised communication was an independent variable with no research question behind it . . . 8

2 Comparison against Knippenberg, Holenderski & Menkovski (2021) **9**
- 2.1 Carried over directly (and appropriately) . . . 9
- 2.2 Correctly, non-trivially adapted . . . 9
- 2.3 Genuine extensions (no counterpart in the source paper) . . . 10
- 2.4 [K1, Deviation] Strong connectivity . . . 10
- 2.5 [K2, Gap] No task deadlines . . . 11
- 2.6 [K3–K6, Deviation] Smaller, favourable deviations . . . 11
- 2.7 [K7, Clarity] Cost-asymmetry sentence was untethered, and $c(v, w)$ was never tied to $c(e)$ . . . 12

3 Comparison against Ma, Li, Kumar & Koenig (2017) **13**

4 Where does this leave Method_and_Implementation.md? **15**

A Notation **15**

How to use this document

`problem_formulation_fixed.tex` is a complete, label-preserving drop-in revision of `problem_formulation.tex`. Every fix there is tagged inline (`% [Gxx]`, `% [Kxx]` or `% [Mxx]`); this document is the rationale for each tag, split into three parts by where the finding came from: Part 1 was found by checking the section against itself (does every symbol get defined, do the claimed properties actually follow); Part 2 was found by checking it against the paper its environment and observation model is adapted from; Part 3 was found by checking it against the paper its MAPD terminology and well-formedness condition are adapted from. Severity key used throughout: **Error** = mathematically wrong as stated; **Gap** = not wrong, but undefined/underspecified where the section otherwise commits to full formality; **Deviation** = a considered, defensible difference from a source, not necessarily a defect; **Clarity** = correct and complete, but hard to read.

1 Internal-consistency review

[Error] The potential-function claim (target-distance annotation)

Original text: "The annotation is a potential: its difference along an edge, $\eta_i(w,t) - \eta_i(v,t)$, is negative exactly on shortest-path moves."

Why it's wrong: by the triangle inequality on d_G,

$$\eta_i(w,t) - \eta_i(v,t) \;\geq\; -c(v,w) \qquad \text{for every } (v,w) \in E,$$

with *equality iff* (v,w) lies on a shortest path from v to $q_i(t)$. "Negative" is necessary for a shortest-path move but not sufficient: a move can strictly satisfy the inequality (decrease η_i by less than $c(v,w)$) while not lying on any shortest path. Concretely: $c(v,w) = 1$, $d_G(v,q) = 5$, and some other route makes $d_G(w,q) = 4.5$ – the difference is -0.5 (negative), but (v,w) is not on a shortest path (that would require $d_G(w,q) = 4$ exactly).

Fix: restated as the inequality with the correct equality condition (`eq:potentialbound` in the fixed .tex); surrounding prose now says a negative difference indicates *progress*, not *optimality*. This is the one place a formally stated property was actually false, not merely underspecified.

[G1] V_{mov} vs. V_{str}/V_{del} – can an agent ever reach a pickup or delivery vertex?

Agents only ever occupy $\ell_i(t) \in V_{mov}$. The original text declares $V_{str}, V_{del} \subseteq V$ (not $\subseteq V_{mov}$) while calling all four roles a "partition" – which $V_{ep} \subseteq V_{mov}$ already contradicts, since a partition requires disjoint blocks. If V_{str} and V_{del} are disjoint from V_{mov}, as "partition" suggests, an agent can never occupy a pickup or delivery vertex, leaving pickup/delivery physically undefined. **Fix:** $V_{str}, V_{del} \subseteq V_{mov}$ now stated explicitly (they must be occupiable), "partition" replaced by "label," and pairwise disjointness explicitly not assumed.

[G2] The task tuple is missing the SKU it names

$\tau_j = (r_j, s_j, g_j)$ in the original, but the coupling condition immediately after it, $x_{r_j}(k_j, s_j) > 0$, uses k_j – the requested SKU – which is never introduced anywhere. The thesis's central equation rests on an undefined symbol. **Fix:** `eq:task` is now a quadruple $\tau_j = (r_j, s_j, g_j, k_j)$, $k_j \in \mathcal{K}$.

[G3] Π and $\mathcal{F}$ are never defined as sets

The formal decision problem optimises over $F \in \mathcal{F}$, $\pi \in \Pi$, but neither space's domain/codomain is ever written down – only *what information* each may condition on (Table I) is described. **Fix:** $\mathcal{F}$ defined as the set of measurable functions $F : \mathcal{X} \times \mathbb{R}_{\geq 0}^{|\mathcal{K}|} \times \mathbb{R}_{\geq 0}^{|E|} \times \mathbb{R}_{\geq 0}^{|E|} \to \mathcal{X}$; Π defined as the set of pairs $\pi = (\pi^{\text{assign}}, \pi^{\text{route}})$ of an assignment map and a routing map with stated domains, constrained only by Table I. A related loose end fixed in the same place: eq. 8 ($x_t \in \arg\min$) can be set-valued while eq. 7 ($x_t = F(\ldots)$) claims a function; the fix adds a tie-breaking-rule requirement so eq. 7 is well-defined.

[G4] No initial storage configuration

$x_t = F(x_{t-\Delta}, \ldots)$ needs x_0 to bootstrap it at $t = \Delta$; x_0 is never introduced. **Fix:** $x_0 \in \mathcal{X}$ stated as supplied with the problem instance.

[G5] $\mathcal{Q}_t, \mathcal{B}_t, \mathcal{C}_t$ are only described narratively

Disjointness is asserted ("a property asserted at runtime") rather than derived, unlike every other object in the section. **Fix:** introduces the assignment time y_j ($r_j \le y_j \le d_j$) and defines

$$\mathcal{Q}_t = \{\tau_j : r_j \le t < y_j\}, \quad \mathcal{B}_t = \{\tau_j : y_j \le t < d_j\}, \quad \mathcal{C}_t = \{\tau_j : d_j \le t\}.$$

Disjointness now follows from the three intervals partitioning $[r_j, \infty)$, rather than being separately asserted.

[G6] The congestion feature $\delta_i(t)$ is defined but never wired in

$\delta_i(t)$ appears in the Learning Problem subsection, but $o_i(t)$ was fixed earlier and never amended, and $\delta_i(t)$ is never connected to R_i either. **Fix:** moved `eq:congestion` into the Observations subsection, right after η_i, and added $\delta_i(t)$ as a third component of `eq:observation`.

[G7] W_T has no formula and an ambiguous relationship to $\hat{w}_t(e)$

W_T appears only inside the system-level criterion, described as having an operational definition deferred to Implementation, with zero conceptual description here – and a name close enough to the already-used per-edge estimate $\hat{w}_t(e)$ that the two could be mistaken for the same object.

First-pass fix (superseded below): an initial revision gave $W_T = \frac{1}{|\mathcal{C}_T|}\sum_{\tau_j \in \mathcal{C}_T}(y_j - r_j)$, mean pre-assignment queueing delay, using y_j from [G5]. This was internally consistent but, once `Method_and_Implementation.md`'s operational suggestion for W_T ("timesteps in which an agent waited or was overridden while holding an active task") was checked against it, turned out to describe a *different* quantity – and queueing delay is largely redundant with the backlog term $B_T = |\mathcal{Q}_T| + |\mathcal{B}_T|$ already in `eq:functional`, the same double-counting concern the section already raises for throughput vs. $\bar{\zeta}_T$.

Current fix: W_T now measures time lost *after* assignment instead, which is not redundant with B_T: for agent i serving τ_j over $t \in [y_j, d_j)$, let $\omega_j(t) \in \{0, 1\}$ indicate agent i is blocked at t (chose wait, or had a move overridden into a wait by the conflict-resolution operator, Section 4/`sec:method:resolution`). Then

$$W_T = \frac{1}{|\mathcal{C}_T|} \sum_{\tau_j \in \mathcal{C}_T} \sum_{t=y_j}^{d_j - 1} \omega_j(t).$$

This version also gives the resolution operator's "overridden" flag (needed anyway for the feasibility mechanism) a direct second use as W_T's data source, rather than requiring separate instrumentation. W_T remains distinct from $\hat{w}_t(e)$ for the same reason as before: one is a realised scoring statistic, the other a forward-looking estimate feeding F.

[G8] Entropy is undefined at the boundary

`eq:entropy` is stated "for $|E_T^+| > 1$" only; at $|E_T^+| \in \{0, 1\}$ the denominator $\log |E_T^+| = 0$. **Fix:** $|E_T^+| = 1$ defined as maximally concentrated by convention ($H_T := 0$, $C_T := 1$); $|E_T^+| = 0$ reported separately, like $|\mathcal{C}_T| = 0$.

[G9] Are constraints (4)–(5) actually constraining anything in the decision problem?

Section B already states vertex/swap-conflict feasibility is enforced by a mechanism applied regardless of which π is chosen. If so, every π satisfies them by construction, and listing them in the arg min over (F, π) doesn't prune the search space. **Fix:** one clarifying paragraph: these constraints are retained to state explicitly that a solution is a claim about *feasible realised trajectories*, unlike `eq:feasiblestorage` and `eq:coupling`, which genuinely do restrict F and the realised task stream.

[G10] Is G directed, and is it connected?

One-way aisles imply G must be directed, but this is never stated; d_G is used pervasively as if always finite, with no connectivity assumption stated. **Fix:** G explicitly stated as directed; a finiteness assumption added (tightened further in Part 2, [K1]).

[Gap] $\mathcal{U}_i$ (POSG action space) vs. $\mathcal{U}(v)$ (vertex-local action set)

`eq:actions` is explicitly vertex-dependent, but the POSG tuple lists a fixed per-agent $\mathcal{U}_i$, with no statement of how one becomes the other. **Fix:** $\mathcal{U}_i := \bigcup_{v \in V_{mov}} \mathcal{U}(v)$, with only $\mathcal{U}(\ell_i(t))$ enabled at a given state (the rest masked to probability zero).

[Gap] Does $J(\theta)$ apply to all three controller architectures?

θ is only introduced for the decentralised architecture, but `eq:objective` is presented as though general. **Fix:** one sentence: it applies to whichever regime is realised as a learned controller, with θ ranging over that regime's own parameters.

[G13] Is the conflict-resolution mechanism itself deterministic?

Section J claims execution is deterministic with P stochastic only through order arrivals, but never confirms the yield-resolution mechanism is itself deterministic – if it used randomised tie-breaking, that would be a second stochasticity source. **Fix:** one sentence confirming it is deterministic given the state.

[G11 / Clarity] Symbol collisions: five unrelated concepts spelled with the letter "C," three with "Q," two with "O"

Not an error, but slows careful reading (eq. 21 has both $\bar{c}_T$ and $\mathcal{C}_T$ in the same line). **Fix:** rather than renaming established symbols this late – Method/Implementation may already reference them, and a rename should touch the whole thesis at once, not one section in isolation – a disambiguation table is provided in Appendix A of *this* document, ready to paste into the thesis's own appendix chapter, with only a one-line pointer left inline in Section IV (per `Method_and_Implementation.md`'s own rule that a full notation table is appendix material).

Left open / not changed

- m **used before being defined** (Section A uses it, Section B defines $m = |\mathcal{A}|$) – fixed with a one-line forward pointer rather than reordering subsections.
- **A full rename of the C/Q/O-family symbols** – flagged via the appendix table rather than executed; out of scope for a Problem-Formulation-only fix.

- **Unit/scale normalisation across the five terms of $\mathcal{J}(F, \pi)$** – summing time, cost, task-count and dimensionless quantities under weights $q_\bullet$ with no stated calibration scheme is a Method-level concern (how $q_\bullet$ are chosen), not a Problem-Formulation error, but worth having an answer ready for: linear scalarisation across unlike units needs either normalisation or explicit justification for the weights.

- **Section H's $u_t \sim \pi(\cdot \mid s_t)$ vs. the decentralised $\pi_\theta(\cdot \mid o_i(t))$** – already reconciled informally ("with the corresponding product of local policies under decentralised execution"); not tightened further.

1.1 [G14] Congestion sensitivity in routing was decentralised-only, which breaks the cross-architecture comparison

The Project Description added to this file (Section **??**) lists "congestion sensitivity: disabled or enabled in the storage and routing layers" as an independent variable, orthogonal to controller architecture (Table **??**). Storage already had this: β in `eq:storageobjective` toggles it. Routing did not: $\delta_i(t)$ `eq:congestion` was defined only for the decentralised agent's local window, so "congestion sensitivity: enabled" had no realisable meaning for the centralised or section-based architectures at all – two of the three levels of the controller-architecture factor could not actually take both levels of the congestion-sensitivity factor. This is not a readability gap; it is a factorial-design defect: RQ2 (does the effect of coupling differ by architecture) and RQ3 (does congestion sensitivity help) cannot both be asked cleanly if one factor is only realisable at one level of the other.

Fix: `eq:congestion`'s formula and label are unchanged (so every existing (**??**) in `method_implementation_skeleton.tex`, i.e. `eq:reward` and `eq:readout`, still resolves identically). A general occupancy fraction $\delta(S, t; \text{excl})$, `eq:occupancy`, is introduced in **??** – the earliest point in the document where all three architectures and the zone partition `eq:partition` are already in scope – and $\delta_i(t)$ is restated as its decentralised instance ($S = V_i^{(r_{\text{cng}})}(t)$, excl $= \{a_i\}$), with the centralised and section-based instances given in prose ($S = V_{\text{mov}}$ and $S = Y_q$ respectively, both with excl $= \emptyset$, since a coordinator is not itself an occupant the way an agent asking about its own neighbourhood is). Concretely realising this signal inside the centralised/section-based planners — a congestion-weighted variant of each, mirroring the storage layer's $F_{\text{dem}}/F_{\text{cng}}$ ladder — is left as a named, forward- pointed Method-level task (**??**), not written here, per this document's own Problem-Formulation/Method boundary.

1.2 [G15] Computational cost was promised in the new Introduction but never formalised

The Introduction and Project Description name "computational cost" / "controller runtime" as an outcome measure and dependent variable, but `eq:functional` had no term of that kind and **??** never defined one. **Fix:** κ_T, `eq:runtime`, mean per-decision wall-clock cost, defined and explicitly excluded from `eq:functional` for the same reason throughput already is (different unit family; a $q_\bullet$ weight would fix an arbitrary seconds-to-quality exchange rate). No Method-file edit was needed for this one: `method_implementation_skeleton.tex`'s **??** already specifies wall-clock logging split by π^{assign} vs π^{route} and training vs inference time – the operational "how" was already in the right place, waiting for a "what" to point at it. `eq:rqformal` is also extended to range over the congestion-sensitivity toggle from [G14], closing the RQ3 gap the same review surfaced.

Correction to prior advice in this review: do not trim ??

An earlier round of this discussion (not in this document) suggested compressing the observation subsection's proof-level detail – `eq:potentialbound`'s equality condition, the congestion radius, the communication graph – to rebalance it against the comparatively short storage/coupling treatment. Re-reading `Method_and_Implementation.md` in full shows this was the wrong prescription: that document's own cut list already checked subsection F against exactly this question ("what Section IV must define is the observation $o_i(t)$ – the induced subgraph, the potential η_i, and the communication graph – because those specify what an agent is allowed to know, which is a property of the problem") and kept it, routing only the encoder/message-passing equations to **??**, where `problem_formulation_fixed.tex` already has them. The size imbalance documented under [G14] above is real and numeric, but it is not evidence of misplaced Method content – there isn't any to move. It reflects that defining what a bounded, local observer may know legitimately takes more apparatus than defining an abstract storage update does; treating that as a defect and cutting it to match a line count would itself be the kind of drift this section is supposed to resist. The right response to the imbalance was narrower than "trim Observations": fix the one place the imbalance was load-bearing, which is the congestion-signal asymmetry above.

[G16] d_G was defined over all of G, not just the occupiable subgraph

$N^+(v) = \{w \in V_{\text{mov}} : (v, w) \in E\}$ already restricts every real move to land in V_{mov}, so no agent trajectory can ever pass through a vertex outside it. But $d_G(v, w)$ was defined as the shortest-path cost "along directed edges of G" – the full graph, V_{mov} included or not. If a future instance ever has $V_{\text{mov}} \subsetneq V$ (the environment section explicitly permits this: $V_{\text{mov}} \subseteq V$, not $=$), d_G could report a shortest-path cost that routes through a non-occupiable waypoint – a number no real agent could ever achieve, since every real hop is constrained to V_{mov}. Every downstream use of d_G (η_i's potential, the connectivity assumption, the communication-graph radius, all of **??**) would then silently be using an inadmissible distance. **Fix:** $d_G(v, w)$ is now defined only for $v, w \in V_{\text{mov}}$, over $G[V_{\text{mov}}]$ (the same induced-subgraph notation `eq:localsubgraph` already uses), with a parenthetical noting the restriction is currently vacuous since every generated instance so far has $V_{\text{mov}} = V$. Found while explaining, not auditing – a reminder that "why does this indirection exist" questions are worth asking even after a section has already been reviewed twice.

[G17] "Free agent" was used but never defined

π^{assign}'s own domain (**??**) is stated as "free agents and $\mathcal{Q}_t$," and `method_implementation_- skeleton.tex`'s `eq:assignmentrule` matches "the free agent" to each task – both use the term as if self-evident, and neither states what makes an agent free rather than occupied. Nothing else in **??** names *which* agent is serving an active task either: y_j (assignment *time*) was defined, but not *to whom*. **Fix:** $\sigma(j) \in \{1, \ldots, m\}$ added alongside y_j in `eq:lifecycle`'s paragraph (the agent task assignment selects at y_j), and `eq:free` defines free/occupied directly from σ and $\mathcal{B}_t$: agent a_i is free at t iff no active task is served by it. `eq:assignmentrule` in the Method skeleton now points back to `eq:free` instead of relying on the same informal term it previously left undefined.

[G18] The decentralised agent's "current target" $q_i(t)$ was named but never tied to the task model

The potential annotation $\eta_i(v, t) = d_G(v, q_i(t))$ (`eq:potential`) introduced $q_i(t)$ only as "the agent's current target," with no link back to s_j, g_j, or σ – the objects **??** already defines for exactly this

purpose. A reader had no way to tell whether $q_i(t)$ meant the pickup vertex before pickup, the delivery vertex after, or something else, and the free-agent case (no task to serve) was left entirely unaddressed. Found during a readability pass over the first eight pages, not an audit – the same pattern as `[G16]`: a symbol used fluently enough that the missing tie to earlier notation is easy to read past. **Fix:** $q_i(t)$ is now defined as s_j before pickup and g_j after, for whichever task τ_j the agent serves ($\sigma(j) = i$); the free-agent case is explicitly named as unresolved here and forward-pointed to **??** rather than left implicit.

[G18, continued] "Before pickup" / "after" needed a mechanism, not just a boundary word

?? (the forward pointer this fix's own text promises) never actually returns to resolve either half of `[G18]`: it covers the GNN encoder, not the pickup transition or the free-agent case. This surfaced while implementing `controllers/centralised.py` (`slap-mapd-coupling`, M2): a routing controller cannot tell whether an active agent's sub-goal is s_j or g_j from $\ell_i(t)$ alone – once the agent leaves s_j, $\ell_i(t) \neq s_j$ again, which a location-only check would misread as "not yet picked up" and send the agent back. **Fix:** a fourth task timestamp, p_j (pickup time), with $y_j \leq p_j \leq d_j$: the first t at which the agent serving τ_j reaches s_j. Then $q_i(t) = s_j$ if $t < p_j$ (or τ_j has not yet been picked up), g_j otherwise, for whichever τ_j is active under $\sigma(j) = i$. p_j sits alongside y_j/d_j in (**??**)'s family of timestamps but is not itself a lifecycle-set boundary – $\mathcal{Q}_t/\mathcal{B}_t/\mathcal{C}_t$ are unchanged by it, since an active task is still just $y_j \leq t < d_j$ regardless of pickup. Implemented as `Task.pickup_time` / `Task.pick_up(t)` / `Task.current_goal`, mirroring how y_j/d_j are already handled. The free-agent case remains genuinely open: this fix only ties down $q_i(t)$ for an agent currently serving a task, not what $q_i(t)$ should be for a free one, which **??** still owes a decision (this is a Method-level design choice – e.g. the agent's own current vertex, a designated rest vertex, or something else – not something the implementation should guess at silently).

1.3 [G19] Three local radii (d_{obs}, r_{cng}, r_{com}) and `[G14]`'s general $\delta(S, t; \text{excl})$ collapsed into one field of view

`[G14]`'s own fix introduced a general occupancy fraction $\delta(S, t; \text{excl})$, `eq:occupancy`, precisely so congestion sensitivity could be realised at every architecture level, with three named instances (whole graph for centralised, one zone for section-based, one neighbourhood, radius r_{cng}, for decentralised). Checked against the actual independent variables in **??**, none of the three local radii this thesis had accumulated – d_{obs} (observation depth), r_{cng} (congestion radius), r_{com} (communication radius) – was ever itself an experimental factor. Congestion sensitivity and communication are both on/off toggles; nothing sweeps their range. So keeping three separately-named, separately-justified radii, and a general S that let the three architectures' congestion readings differ in kind (a single whole-graph average for centralised versus a per-neighbourhood reading for decentralised) as well as in scope, bought generality nothing in this study's design actually uses, and the whole-graph instance specifically meant a centralised controller had no way to act on congestion in any particular region, only a single system-wide average. Found while checking this against **?** (CRAMP), whose crowd-density feature uses exactly one local window for vision, crowd-density, and inter-agent communication at once, no separate range for each. **Fix:** r_{cng} and r_{com} are removed; d_{obs}'s induced subgraph (`eq:localsubgraph`, unchanged in form) is renamed *field of view* in prose and moved from **??** into **??**, since it is now a graph primitive reused everywhere, not a decentralised-only bound. `eq:commgraph`'s edge condition now reads $d_G(\ell_i(t), \ell_j(t)) \leq d_{\text{obs}}$. `[G14]`'s general $\delta(S, t; \text{excl})$ and its three-instance description list are both removed; `eq:congestion` is now

the one definition, $\delta_i(t)$, the occupancy fraction of any agent i's own field of view, with nothing architecture-specific written into it at all. Which agents' $\delta_j(t)$ a given controller gets to read is no longer a congestion-specific rule, it falls out of Table **??** applied to one more quantity in the state (decentralised: its own only; section-based: every agent in its own zone; centralised: every agent). A section-based agent's field of view is explicitly *not* clipped at its own zone boundary: Table **??** already lists "boundary protocol" as part of what section-based conditions on, and clipping every local reading exactly at the boundary would blind that protocol at exactly the place it exists to help. This does not reopen `[G14]`: congestion sensitivity remains realisable at every architecture level, just via one narrower, comparable feature instead of three differently-scoped ones. `fig:congestion` (`congestion_mini.py`) and `fig:observation` (`observation.py`) were regenerated to match; `2.Notation.tex` and `4.Implementation.tex`'s hyperparameter table had their $r_{\text{cng}}/r_{\text{com}}$ rows removed.

1.4 [G20] Decentralised communication was an independent variable with no research question behind it

Aim and Scope (**??**) lists "communication: absent or local for the decentralised controller" as independent variable #4. None of RQ1-RQ4 (**??**) mention communication, and `eq:rqformal`, which restates every symbol the RQs formally range over (the information regime via `tab:information`, congestion sensitivity via β and `eq:congestion`, load via m and λ_{task}), omits it too. The one sentence that would tie it to a purpose is commented out directly after RQ4: "Local communication is treated as an ablation within the decentralised architecture... testing whether communication moderates congestion under partial observability." Not visible in the compiled document. So the feature (`eq:commgraph`, a subsubsection, a table row, a figure legend note, and a clarifying paragraph distinguishing it from $\delta_i(t)$) currently has no stated research question, despite being real, working code in `slap-mapd-coupling` (`gnn_encoder.py`, `observation.py`, `rl_module.py`, `experiment_config.py`, `controllers/decentralised.py`), with message content already concretely specified in `4.Implementation.tex` ($(d_G(\ell_i(t), \ell_j(t)),\ \eta_j(\ell_j(t), t))$ per message) and a free ablation ($Q = 0$ collapses to no message passing). **Fix:** given a research question rather than removed. A conditional **RQ5** was added after RQ4 in **??**: "Does local communication improve decentralised routing beyond field-of-view sensing alone, and does that benefit depend on how well hop-adjacency predicts route overlap in this thesis's non-lattice graph? Addressed after RQ1-RQ4 if time permits. If not reached, communication remains implemented but unevaluated." `eq:rqformal` now notes the communication term is conditional and not required for the main factorial design. **??** states explicitly that if RQ5 is not reached, communication stays in the model as an unexercised capability, not a claim the results support. The previously commented-out sentence in Aim and Scope is now active prose reflecting the conditional framing. No code change was needed, $Q = 0$ already gives the free ablation. Chosen over outright removal because the non-lattice angle (hop-adjacency not reliably predicting route overlap, unlike in **?**'s lattice world) is a genuinely different question CRAMP itself cannot test, so it was worth keeping as a deliberately conditional, not required, question rather than cutting real working code for a question with no examiner-visible cost to leaving unanswered. Full argument in `communication_scope_decision.md` at the workspace root.

Editorial: forward references resolved during the first-eight-pages readability pass

Three further cases of a symbol used before its own definition were found and fixed the same way as the existing m item above – a forward pointer or a plain-words description at the early use-site,

no reordering of subsections:

- $V_i^{(r_{\text{cng}})}(t)$ **used before its own definition** – **??** named the decentralised congestion window by its full notation before **??** defines it. Fixed by describing the window in words at first use and leaving the notation itself only at its actual definition site.
- $N^+(v)$ **named before** `eq:actions` **defines it** – **??**'s distance discussion named the successor set a few lines before its formal definition. Fixed by describing the restriction in words and pointing forward to `eq:actions` instead of naming the symbol early.
- W_T **cited in ??, several subsections before** `eq:waiting` **defines it in ??** – fixed with an explicit "defined in Section **??**" pointer at first use, matching the convention the m item above already relies on.

2 Comparison against Knippenberg, Holenderski & Menkovski (2021)

van Knippenberg et al. (2021) ("Time-Constrained Multi-Agent Path Finding in Non-Lattice Graphs with Deep Reinforcement Learning," ACML 2021) is the closest prior-art paper to this thesis's environment and observation model: a directed, non-lattice graph; a local observation of fixed depth d centred on the agent; a per-node distance-to-goal annotation; a shared policy over a wait-or-move action space. The thesis's problem formulation is best read as taking that architecture and (a) embedding it inside a lifelong/online MAPD setting with continuous task arrivals rather than van Knippenberg et al.'s batch/episodic one-job-per-agent setting, and (b) coupling it to a new second layer – storage placement – that has no counterpart in van Knippenberg et al. at all. What follows is organised by what carried over unchanged, what was correctly adapted, what is a new extension, and one place the comparison surfaces an actual gap.

2.1 Carried over directly (and appropriately)

- **Directed, non-lattice graph.** Identical framing in both.
- **Local observation of fixed depth d.** van Knippenberg et al.'s observation is "centered on the agent's current location" with size "determined by a global depth parameter d"; this is exactly $V_i^{(d)}(t)$, `eq:localsubgraph`.
- **Distance-to-goal node annotation.** van Knippenberg et al. includes, per visible node, "the distance of each node to the goal node," precomputed once per instance – this is exactly $\eta_i(v,t) = d_G(v, q_i(t))$, `eq:potential`.
- **Wait-or-move action space, shared policy.** Identical in both.

2.2 Correctly, non-trivially adapted

- **Dynamic vs. static target.** van Knippenberg et al.'s distance-to-goal values "need only be pre-computed once for every problem instance," because each agent has exactly one job (one fixed goal) for the whole episode. The thesis is lifelong: an agent's target changes at every reassignment, and the thesis states this explicitly ("η_i itself changes whenever the target changes, which happens at every reassignment"). This is a correct and necessary generalisation the source paper didn't need to make – a good sign, not a borrowed shortcut.

- **Vertex/swap-conflict as two orthogonal equations vs. one prose definition.** van Knippenberg et al. defines a "node collision" (two cases in prose: moving onto an occupied node, or two agents moving to the same node) and an "edge collision" (explicitly qualified to *bi-directional* edges only). The thesis's `eq:vertexconflict` covers both node-collision cases with one inequality, and its `eq:swapconflict` needs no "bi-directional" qualifier: a swap is only physically reachable when `eq:transition` already requires both $(v, w) \in E$ and $(w, v) \in E$ to hold for the two agents involved, so the bidirectionality falls out of constraint composition instead of being hand-added to the collision definition. This is a tighter formalisation than the source.

2.3 Genuine extensions (no counterpart in the source paper)

- **The entire storage layer** – x_t, $\mathcal{X}$, `eq:storageupdate`, `eq:storageobjective`, and the coupling condition `eq:coupling` that links it back to routing. van Knippenberg et al. is a single-layer routing problem; this second, slower-timescale layer and its feedback loop is the thesis's actual contribution over the source paper, and is worth stating this plainly wherever the thesis positions itself against MAPF/MAPD literature.

- **Explicit communication graph $G_t^{\mathcal{A}}$ and messages.** van Knippenberg et al. states explicitly they consider only "implicit communication" (agents observe each other's state, but nothing is explicitly communicated). `eq:commgraph` and the message component of `eq:observation` are a genuine addition.

- **The congestion feature $\delta_i(t)$.** No counterpart in the source.

- **Three information regimes (centralised / section-based / decentralised) with a shared assignment mechanism.** van Knippenberg et al. studies a single decentralised regime and has no assignment sub-problem at all (each agent's one job is given, not chosen online) – the thesis's Table I and the assignment/routing split are new, driven by the lifelong setting's need to keep deciding who does what.

- **Throughput, backlog, entropy/concentration.** All specific to the continuous/lifelong framing; van Knippenberg et al.'s objective (a single scalar per solved batch instance, eq. 1–2 in the source) has no analogue for an ongoing system with no end state.

2.4 [K1, Deviation] Strong connectivity

van Knippenberg et al. state plainly: "environment graphs are assumed to be strongly connected, i.e. each node is reachable from all other nodes." The thesis (even after [G10]'s fix) states the weaker "$d_G(v, w)$ is finite for every pair of vertices that appears in the constructions below." The source's version is simpler to state and to verify computationally (one graph-level check, independent of what gets used later); the thesis's version is strictly weaker and therefore more permissive of the "irregular blocks" / unreachable side-room layouts its own motivating paragraph gestures at. **This is a judgement call, not a correctness issue**: if generated instances never actually exercise a disconnected pocket, the weaker statement buys nothing and the source's crisper assumption should probably be adopted outright; if partial reachability is a deliberate stress case for the decentralised controller (an agent whose target briefly becomes unreachable), the weaker version is doing real work and is worth keeping, explicitly framed as a deviation from the more common assumption. The fixed .tex takes the conservative option – keeps the weaker statement, adds one sentence contrasting

it with van Knippenberg et al.'s choice and naming the tradeoff – and leaves the actual decision to Section VI (*Instance generation*), where it is checkable against what instances are actually generated.

2.5 [K2, Gap] No task deadlines

This is the most substantial finding from the comparison. Time constraints are van Knippenberg et al.'s entire contribution – their title is literally "Time-*Constrained* MAPF" – realised as a job 4-tuple $(t_i^s, t_i^d, v_i^s, v_i^g)$ with an explicit deadline t_i^d, and an objective that is either a hard feasibility cut (a plan that misses its deadline is invalid, eq. 1 in the source) or a soft penalty term $w \cdot \max(0, t_{\pi,\text{actual}} - t_{\pi,\text{deadline}})$ added to the cost (eq. 2). The thesis's task tuple `eq:task` carries only a release time r_j; there is no deadline anywhere, and timeliness is scored only in aggregate, after the fact, via the mean $\bar{\zeta}_T$ (`eq:throughput`). Under the thesis's formulation, no individual task can be said to have *failed* – an arbitrarily late delivery only nudges an average.

Given the thesis explicitly reuses van Knippenberg et al.'s architecture (built specifically to handle deadlines) while dropping the one thing that architecture was built for, this needed to be an explicit, stated choice rather than a silent omission – warehouse SLAs are a real business constraint, so a reader familiar with the source paper would reasonably ask why deadlines didn't carry over too. **Fix (already applied in `problem_formulation_fixed.tex`, Section J):** one paragraph stating the omission explicitly, alongside the other scope exclusions, and naming what reintroducing deadlines would require: a t_j^{d} field added to `eq:task`, plus either a feasibility condition (hard) or a term in `eq:functional` of the same shape as van Knippenberg et al.'s eq. 2 (soft). Both are Problem-Formulation-level changes, not Method-level ones, since either would redefine what a valid or well-scored solution *is* – consistent with how the thesis already treats `eq:coupling` and the conflict constraints as problem-level because they define validity, not merely score it.

2.6 [K3–K6, Deviation] Smaller, favourable deviations

- **[K3] Edge cost $c(e) > 0$ (thesis) vs. $c(v, v') \geq 0$ (source).** The thesis's strict positivity rules out zero-cost cycles, which keeps d_G well-behaved as a strictly hop-sensitive quantity and keeps the corrected potential-bound argument ([Error] above) free of degenerate edge cases. A defensible tightening, not a regression.

- **[K4] No `canWait`-style per-node restriction.** van Knippenberg et al. give each node a boolean attribute controlling whether waiting is allowed there (e.g. a mainline track where a train cannot simply stop); the thesis assumes waiting is legal everywhere via a single c_{wait}. This underwrites the thesis's "a collision-free joint action always exists" claim (Section B) for free, but is a real loss of expressiveness relative to the source if a future revision wants to model genuinely non-stoppable segments – worth a one-line note in Scope if that generality is ever wanted; not fixed here since it is not currently claimed as a capability.

- **[K5] No dummy-node start-time mechanism needed.** van Knippenberg et al. materialise each agent at a private dummy node until its individual start time t_i^s elapses, to avoid an agent appearing mid-graph and causing a spurious collision. The thesis avoids this entirely by construction: the fleet $\mathcal{A}$ is fixed and persistent (agents idle at V_{ep} between tasks), and it is *tasks*, not agents, that have release times r_j. A structural consequence of the lifelong framing, not a borrowed mechanism – correctly absent, not simply omitted.

- **[K6] Reward design (Method-level, noted here for completeness).** van Knippenberg et al. use a fixed table of empirically-tuned per-action penalties (Table 1 in the source:

e.g. move -0.1, collision -2, goal completion $+10$) with no stated invariance guarantee. `Method_and_Implementation.md` instead specifies potential-based shaping, $\gamma\Phi_i(t{+}1) - \Phi_i(t)$ with $\Phi_i(t) = -\eta_i(\ell_i(t), t)$, which provably preserves the underlying game's equilibria (Ng et al., 1999; extended to dynamic potentials by Devlin and Kudenko, 2012) – directly relevant here since Φ_i is non-stationary by construction (the target $q_i(t)$ changes at every reassignment). van Knippenberg et al. even flag their own use of DQN as a limitation and name PPO as future work (their Section 8); `Method_and_Implementation.md`'s planned PPO training procedure directly answers that. Correctly scoped to Method, not Problem Formulation – noted here only because the comparison is what surfaced it as a deliberate, citable improvement rather than an arbitrary choice.

2.7 [K7, Clarity] Cost-asymmetry sentence was untethered, and $c(v, w)$ was never tied to $c(e)$

An earlier revision of `problem_formulation_fixed.tex` added a standalone sentence under the `[G10]` tag stating that $(v, w) \in E$ does not imply $(w, v) \in E$ and that the two directions "may carry different costs." Directional cost asymmetry is not wrong – van Knippenberg et al.'s own formalism defines cost per ordered pair $c(v, v')$ on a directed graph too – but it is a free corollary of directedness, not an independent modelling choice, and no generated instance in either this thesis or van Knippenberg et al. actually exercises a bidirectional edge with unequal costs in each direction (their bidirectional edges are drawn as a single shared cost, e.g. Figure 1's "cost = 1"). The sentence was therefore carrying a `[G10]` tag without matching `[G10]`'s own rationale in this document, i.e. an undocumented claim. **Fix:** folded into the directedness explanation itself ("directedness matters concretely here ... aisles are often one-way, so $(v, w) \in E$ does not by itself imply $(w, v) \in E$, nor does it imply the two directions cost the same where both exist"), so the asymmetry reads as a consequence of being directed rather than a separately asserted feature.

Separately, $c(v, w)$ was used three times (in `eq:onestepcost` and `eq:potentialbound`) without ever being identified with $c(e)$ for $e = (v, w)$, the only place the cost function is actually defined – a gap the section's own notation-disambiguation paragraph (`sec:pf:notation`) did not catch, since it lists $c(e)$, $\hat{c}(v, w)$, $\bar{c}_T$, C_T and $\mathcal{C}_t$ as the family needing disambiguation but never states that plain $c(v, w)$ *is* $c(e)$. **Fix:** one clause added where $c(e)$ is first defined – "written $c(v, w)$ when $e = (v, w)$" – closing the gap without introducing a sixth symbol into the disambiguation table.

[K8, Deviation] Action masking vs. Knippenberg's penalised-invalid-move handling

Re-reading §3 of van Knippenberg et al. closely: "If an agent chooses a move action that is not valid, i.e. a move action outside the range of outgoing edges of its current location, the action is treated as a 'wait' action, and it is penalised accordingly." Their fixed-size action head can select a nonexistent edge; the environment silently reinterprets that choice as a wait and lets the reward signal teach the policy not to do that. This thesis's `eq:mask` (`method_implementation_skeleton.tex`, **??**) instead sets the logits for nonexistent moves to $-\infty$ before the softmax, so an illegal move has probability zero rather than a penalised nonzero one – legal *by construction*, not learned by penalty. This was already the right design and already correctly cross-tied to $\mathcal{U}(v)$ vs. $\mathcal{U}_i$ in **??** (the `[Gap]` entry above), but the comparison to van Knippenberg et al.'s own mechanism – and the actual argument for why masking is preferable, not just different – lived only in this document, not in the thesis text itself. **Fix:** two sentences added to the masking paragraph naming van Knippenberg et al. (2021)'s penalised approach explicitly and stating why masking is preferred: it never spends

training signal on moves already known illegal from $N^+(\ell_i(t))$, it keeps the executed action identical to the sampled one (their reinterpretation lets a sampled "move" execute as "wait," which the value function then has to reconcile), and it costs nothing extra since $N^+(v)$ is already required to define $\mathcal{U}(v)$ regardless. Same lesson as `[K6]`: a citable improvement over the source is only citable if the thesis actually says so.

3 Comparison against Ma, Li, Kumar & Koenig (2017)

Ma et al. (2017) ("Lifelong Multi-Agent Path Finding for Online Pickup and Delivery Tasks," AAMAS 2017) is where the term MAPD and the notion of a well-formed MAPD instance both come from; this thesis uses both throughout without, until now, ever citing the source or fully carrying over what it defines.

[M1, Gap] MAPD and well-formedness cited but not attributed

Two uncredited borrowings: "Multi-Agent Pickup and Delivery (MAPD)" at its first use in the Introduction, and the well-formed-instances condition in **??** (at least m non-task endpoints; no path between two endpoints blocked by a third), which is Ma et al.'s Definition 1, restated in this thesis's own words but previously with no citation at all. **Fix:** (Ma et al., 2017) added at MAPD's first use; the well-formedness paragraph now reads "Ma et al. (2017)'s well-formed MAPD instances" rather than stating the condition as though it were invented here. A new three-panel figure, `fig:wellformed`, mirrors Ma et al.'s own Figure 2 (which contrasts a well-formed instance against two ways the condition can fail) rather than inventing a new pedagogical device: (a) well-formed, (b) too few non-task endpoints, (c) a path between two endpoints blocked by a third.

[M1, continued] Re-tasking: permitted in the source, silently absent here

Ma et al. explicitly allow an agent to be re-tasked – reassigned to a different job – any time before it reaches its current task's pickup location, and give the reason: "agents can often be re-tasked before they have picked up a good but have to deliver it afterward." This thesis's `eq:lifecycle` moves a task from $\mathcal{Q}_t$ to $\mathcal{B}_t$ at y_j with no path back, which *implies* no re-tasking, but nothing said so before now. Given the source paper is cited specifically for the MAPD problem class, a reader who knows it would reasonably assume re-tasking carried over too. **Fix:** one paragraph added to **??** stating the narrowing explicitly, using the σ notation from [G17]: $\sigma(j)$ is fixed for the lifetime of $\mathcal{B}_t$-membership, so re-tasking is a Problem-Formulation-level change (it would let π^{assign} revise σ for already-assigned, not-yet-picked-up tasks), not a Method-level one – consistent with how deadlines ([K2]) are already treated as the same kind of change.

[M2, Gap] The restated well-formedness clause checked a narrower endpoint set than the source's

Ma et al.'s own V_{ep} already contains every pickup and delivery location ($V_{\text{tsk}} \subseteq V_{\text{ep}}$), so their path-condition clause (c) – "for any two endpoints, a path exists that traverses no other endpoints" – ranges over task *and* non-task endpoints together. This thesis keeps V_{str}, V_{del} and V_{ep} as three separate, typically-disjoint roles (a deliberate and more general choice than the source's nesting, discussed on its own merits elsewhere), which means the restated condition – "between any pair of endpoints some path avoids every third endpoint" – silently ranged over a narrower set than Ma et al.'s original if "endpoint" were read as this section's V_{ep} alone: it said nothing about a path

between, say, two storage vertices, even though the source's completeness guarantee needs exactly that. **Fix:** one clause added stating that for this specific check, "endpoint" means $V_{\text{ep}} \cup V_{\text{str}} \cup V_{\text{del}}$ together, matching Ma et al.'s convention for exactly the well- formedness test and nowhere else – V_{str}, V_{del} and V_{ep} remain separate roles everywhere else in the section, so this does not quietly redefine any of the three. Found by working through what "more general than the source, not more limited" actually requires of a condition borrowed *from* the source: generality in the role structure does not automatically carry over to a check stated in terms of those roles unless it is checked.

[M3, Gap] "A path exists" needs a direction on a directed graph

Ma et al.'s graph is undirected, so their clause (c) – "a path exists between any two endpoints" – is automatically symmetric: a path from v to w *is* a path from w to v, walked backwards. This thesis's G is directed (**??**, and the whole point of the [G10]/[K1] departure from Knippenberg's own graph choice), so the same sentence is ambiguous here: does the check need a path from v to w, from w to v, or both? Since Token Passing-style algorithms route an agent between whichever pair of endpoints a task assignment happens to produce – not a fixed direction chosen in advance – only requiring *both* directions preserves the completeness guarantee the citation is for; a single-direction reading would silently be a weaker (and wrong) condition. **Fix:** the path-condition clause now states explicitly that, because G is directed, it requires a path from v to w *and* a path from w to v for every pair, each avoiding every third endpoint. Found in the same conversation as [M2] and for the same underlying reason: a property proved on the source's graph does not automatically transfer to a strictly more general graph class without checking which of the source's implicit assumptions (here, undirectedness) the property actually used.

[M4, Deviation] The active phase starts at assignment, not pickup arrival, and this thesis never said so

Ma et al. state plainly: "When the agent reaches the pickup location, it starts to execute the task and the task is removed from $\mathcal{T}$" – their task set only drops a task, i.e. their active phase only begins, at *pickup arrival*, not at assignment. This thesis's `eq:lifecycle` already uses a different boundary – $\mathcal{B}_t$ begins at y_j (assignment) – and the surrounding prose already explicitly rejects "arrival of the agent at s_j" as the boundary. What it did not do, before now, is note that the rejected alternative is Ma et al.'s own convention, or say why this thesis picks the earlier one. Given the source is now cited repeatedly for this exact lifecycle structure, a reader who knows it would notice the silent divergence. **Fix:** one clause naming Ma et al. (2017)'s convention and the reason for departing from it: using y_j rather than pickup arrival means time lost travelling to s_j is attributed to the active phase and is therefore eligible for the blocking measure W_T (**??**), instead of being folded into generic pre-assignment queueing (which B_T's $|\mathcal{Q}_T|$ term already covers). This is a considered choice, not an oversight – but it was an unstated one until this pass.

[M4, continued] This same boundary is why `[G17]` and `[M1]` agree with each other, not just with the source

Re-checking the section once more surfaces that `[G17]`'s `eq:free` is built directly on $\mathcal{B}_t$, which this entry just pinned to y_j. So an agent becomes *occupied* the instant it is assigned, not when it reaches the pickup – and that is precisely why `[M1]`'s re-tasking prohibition holds *before* pickup as well as after: an assigned-but-travelling agent is already occupied under this thesis's boundary, so there is no "free" window in which Ma et al.'s reassignment could happen even if it were permitted. In their model, the later (pickup-arrival) boundary is exactly what leaves that window open, which is why

they can allow it and this thesis's own choice of boundary is why it cannot. [G17], [M1] and [M4] are not three independent findings that happen to be consistent – they are one decision (where $\mathcal{B}_t$ begins) examined from the free/occupied side, the re-tasking side, and the W_T-measurement side respectively. No further fix: the three passages are already individually correct and mutually consistent; this is a note for whoever reads GAPS.tex next; so the connection does not have to be rediscovered from scratch.

4 Where does this leave Method_and_Implementation.md?

Checking problem_formulation.tex (as given, not a hypothetical denser draft) against Method_and_Implementati own six-item cut list:

1. *Controller-architecture algorithmic detail (prioritised planning, matching costs, zone hand-off protocols)* – **not present.** Section E as given only states the abstract information signature and Table I; no algorithm-level content to cut.
2. *Encoder / message-passing equations ($z_i = \mathrm{Enc}_G(\cdot)$, ϕ_q, ψ_q)* – **not present.** Section F as given defines only $V_i^{(d)}$, η_i, δ_i, $G_t^{\mathcal{A}}$, $o_i(t)$ – no architecture equations.
3. *Exact functional forms of D, K, R and weights α, β, χ* – **already deferred** in the text as given ("specified in Section **??**").
4. *The reward function* – **already deferred** (eq:objective's prose: "R_i are left unspecified here").
5. *Defensive justification passages* – **present, low priority.** These exist (paraphrased in Method_and_Implement rather than quoted verbatim) but are one-line footnote-style asides, not misplaced algorithmic content; rephrasing them is a copy-edit, not a structural fix, and wasn't applied here.
6. *Full notation table → appendix* – **applies, but only to the table added in this review** (the original had none). Fixed: see [G11] above and Appendix A below.

The short version: **almost nothing from this review belongs in Method_and_Implementation.md.** Every fix in problem_formulation_fixed.tex is definitional ("what the object is"), which is exactly what Section IV is supposed to contain per that document's own contract table. The one piece of content that needed to move doesn't belong in Method either – by Method_and_Implementation.md's own instruction, a full notation table is *Appendix* material, a third location distinct from both Problem Formulation and Method. That relocation is done in Appendix A of this document (ready to paste into the thesis's own appendix chapter, which is not a file present in this repository).

A Notation

Two tables. The first is the full glossary requested during review: every symbol introduced in Section IV, in the order it first appears, with a plain-language gloss and where it is defined. The second (unchanged from the earlier version of this appendix) is the narrower disambiguation table for symbol families that share a base letter across unrelated objects – keep both, since the first is for looking a symbol up and the second is for the specific worry "is this the same c as that c."
As with the disambiguation table below, replace each eq. (xxx) with \eqref{eq:xxx} when pasting this into the thesis's own appendix – labels already match problem_formulation_fixed.tex verbatim.

Several symbol families in Section IV share a base letter across unrelated objects. None are typos or the same object under different names; each pair below denotes two genuinely different things that happen to look alike.

When pasted into the thesis's actual appendix, replace each `eq. (xxx)` above with `\eqref{eq:xxx}` – the labels already match `problem_formulation_fixed.tex` verbatim, so this table can be dropped in without renumbering anything.

References

Marijn van Knippenberg, Mike Holenderski, and Vlado Menkovski. Time-Constrained Multi-Agent Path Finding in Non-Lattice Graphs with Deep Reinforcement Learning. In *Proceedings of the 13th Asian Conference on Machine Learning*, volume 157 of *PMLR*, 2021.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *ICML*, 1999.

Sam Michael Devlin and Daniel Kudenko. Dynamic Potential-Based Reward Shaping. In *AAMAS*, 2012.

Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong Multi-Agent Path Finding for Online Pickup and Delivery Tasks. In *Proceedings of the 16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 837–845, 2017.

Table 1: Every symbol used in Section IV, in order of first appearance.

Symbol	Meaning (first defined at)
The environment (??)	
$G = (V, E)$	the directed warehouse graph
V_{mov}	vertices an agent may occupy
V_{str}	storage faces (pickup locations)
V_{del}	delivery points
V_{ep}	endpoints: safe to idle at without blocking transit
$c(e) = c(v, w)$	cost of traversing edge $e = (v, w)$, eq. (env def.)
$d_G(v, w)$	cheapest directed path cost, over $G[V_{\text{mov}}]$ only
$N^+(v)$	movable successors of v
$\mathcal{U}(v)$	legal actions at vertex v, eq. (actions)
c_{wait}	cost of waiting one timestep
$\hat{c}(v, w)$	realised one-step cost (handles waiting too), eq. (onestepcost)
$m = \lvert\mathcal{A}\rvert$	fleet size (defined in **??**, used here)
Agents and collisions (??)	
$\mathcal{A} = \{a_1, \ldots, a_m\}$	the fleet
$\ell_i(t)$	agent a_i's location at time t
$u_i(t)$	agent a_i's chosen action at time t
p_i	agent a_i's realised route over the horizon
Where the items are (??)	
$\mathcal{K}$	set of SKU types
x_t	storage configuration: units of each SKU at each storage vertex
b_k	shelf capacity one unit of SKU k consumes
$\text{cap}(v)$	shelf capacity of storage vertex v
$\mathcal{X}$	feasible storage configurations, eq. (feasiblestorage)
x_0	initial storage configuration, supplied with the instance
F	the storage update rule
Δ	storage epoch length (the slow clock's period)
$\hat{\rho}_t, \hat{\mu}_t, \hat{w}_t$	causal estimates: demand, edge traffic, waiting
F_{fix}	the do-nothing baseline, $F_{\text{fix}}(x, \cdot) = x$
$\mathcal{F}$	the set of admissible storage rules
D, K, R	storage objective terms: access distance, congestion, reassignment
α, β, χ	weights on D, K, R in eq. (storageobjective)
Where the work comes from (??)	
P_{ord}	the order-arrival distribution
$\tau_j = (r_j, s_j, g_j, k_j)$	a task: release time, pickup, delivery, requested SKU
λ_{task}	expected tasks released per timestep
y_j	assignment time of τ_j
$\sigma(j)$	the agent assigned to τ_j at y_j
d_j	completion (delivery) time of τ_j
$\mathcal{Q}_t, \mathcal{B}_t, \mathcal{C}_t$	waiting / active / completed task sets, eq. (lifecycle)
ζ_j	service time of τ_j, eq. (servicetime)
Who decides (??)	
$\pi = (\pi^{\text{assign}}, \pi^{\text{route}})$	a controller: assignment map + routing map
Π	the set of admissible controllers
$\mathcal{Y} = \{Y_1, \ldots, Y_p\}$	the zone partition for section-based control
$\delta(S, t; \text{excl})$	occupancy fraction of window S, excluding agents in excl
θ	a learned controller's parameters
What a decentralised agent sees (??)	

Table 2: Symbols that share a base letter but denote unrelated objects.

Symbols	Distinct meanings
$c(e)$, c_{wait}	edge cost / wait cost (instance parameters)
$\hat{c}(v, w)$	realised one-step cost, eq. (onestepcost)
$\bar{c}_T$	mean movement cost per completed task, eq. (movementcost)
C_T	concentration index, eq. (concentration)
$\mathcal{C}_t$	completed-task set, eq. (lifecycle)
$\mathcal{Q}_t$	waiting-task set, eq. (lifecycle)
$q_i(t)$	agent i's current target vertex, eq. (potential)
$q_{\text{svc}}, q_{\text{mov}}, q_{\text{wait}}, q_{\text{conc}}, q_{\text{back}}$	weights in the system-level criterion, eq. (functional)
$\mathcal{O}_i$	observation space, eq. (posg)
O_i	observation function, eq. (posg)
$\mathcal{U}(v)$	action set at vertex v, eq. (actions)
$\mathcal{U}_i = \bigcup_v \mathcal{U}(v)$	agent i's fixed action space in $\mathcal{M}$, eq. (posg)